Lecture 18

Searching

- Searching is used to find specific element from a group of data elements. In our context we are going to study searching in an array.
- There are two algorithms most commonly used to search in an array

1. Linear Search
2. Binary Search

Linear Search

- In linear Search we iterate over all the elements of the array and check if the current element is equal to the target element. If yes, then we return the index of the current element. If we traversed the entire array and the element is not found then we will return -1 as a symbol that element not found.

Algorithm

```
linearSearch -> array, size, target:
     i = 0;
     for i -> 0 to i -> size - 1:
          if arr[i] == target:
                    return i;
    return -1;
```

Time & Space complexity

Best Case: target may be present at the first index. Therefore time complexity is O(1).

Worst Case: target may be present at the last index. Therefore the time complexity is O(n). where n is the size of the array.

- There is no extra memory is taken for searching therefore **space complexity** is constant i.e., O(1).

Application of Linear Search

- When array is small Linear Search is preferred over Binary Search
- When we have unsorted array Linear Search is used to find the desired element.
- Linked List can be traversed linearly. Therefore we cannot apply other sorting algorithm in it except Linear Search.
- Linear Search is easier to implement as compared to Binary Search

Advantages & Disadvantages of Linear Search

Advantages

- Linear Search can be used irrespective of array is sorted or not sorted.
- It is well suited for small data set.

Disadvantages

- Linear Search has time complexity of O(n). Which so slow for large data set.

Binary Search

- Binary Search operates on sorted or monotonic search space. It repeatedly divides array into halves to find the target element.
- Since our problem space each time is getting reduced by halves the time complexity of this algorithm is $O(\log_2 n)$. Base 2 signifies that each time problem is reduced by half.
- You can only apply Binary Search when given problem is sorted otherwise binary search cannot be applied.

Algorithm

```
binarySearch -> array, size, target:
    start = 0
    end = size - 1
    while start <= end:
        mid = start + (end - start)/2
        if array[mid] == target:
            return mid
        if array[mid] < target:
            start = mid + 1
        else:
            end = mid - 1
    return -1
```